Python and Jupyter Setup

Trainer Reference Guide

Purpose and outcomes

Prepare a reproducible Windows-first environment. Learners verify Python 3.11+, create an isolated virtual environment, install dependencies, run a script, and execute a notebook from top to bottom.

Project and exercise walkthrough

- environment_check.py reports interpreter path/version and detects FastAPI, SQLAlchemy, Pydantic, Jinja2, and pytest. Ask learners why the executable path matters.
- 00_python_setup.ipynb introduces cell execution, variables, f-strings, and kernel state. Guided work customizes a greeting; the challenge reports Python and operating-system information.
- Validation: run the environment check and Restart Kernel / Run All. A pass means the expected interpreter and dependencies are visible.

Common difficulties and coaching

Windows may use the Microsoft Store alias or block activation scripts. Demonstrate ‘python -m pip‘ to bind pip to the correct interpreter. Explain that notebook cells share state and hidden order is dangerous.

Project connection

The same environment launches every script, notebook, test, and the final Uvicorn server.

Delivery pattern

1. Explain the idea briefly and ask learners to predict a result.
2. Run the simple example, then connect it to an employee/user workflow.
3. Pair learners for the guided task; require an individual attempt next.
4. Use the challenge as the cumulative project increment.
5. Review the quiz and ask learners to explain, not only show, their code.

Trainer key points

- Keep live coding visible and deliberate. Normalize reading tracebacks.
- Ask what type and value each expression produces before running it.
- Let TODO files remain student-owned; open solutions only during review.
- Check boundary and failure cases, not only the happy path.
- End by tracing how the topic appears in the final portal.